

- RAG Learning

2.2 Problem Solving and Programming

Computer Science — OCR — Component 02 (Algorithms and Programming)

This topic links the theory of computational thinking (2.1) to the practice of writing, structuring, debugging and reasoning about code. It has two strands: **2.2.1 Programming techniques** (the tools a programmer uses: constructs, scope, subroutines, recursion, OOP, IDEs) and **2.2.2 Computational methods** (the strategies used to tackle problems: divide & conquer, backtracking, heuristics, data mining, pipelining, visualisation). Depth and clear terminology matter — examiners reward precise definitions and worked examples.

2.2.1 Programming Techniques

Programming Constructs

All imperative programs can be built from three fundamental constructs, identified by Böhm and Jacopini (1966): **sequence**, **selection (branching)** and **iteration**. These are sufficient (in a Turing-complete sense) to express any computable algorithm.

A **programming construct** is a syntactic structure that controls the flow of execution. The three core constructs are **sequence** (one statement after another), **selection** (choose between paths based on a condition), and **iteration** (repeat a block while a condition holds or for a fixed count).

Sequence

Sequence is the default: statements execute top-to-bottom in the order written. Each statement runs exactly once, and the effects of earlier statements are visible to later ones. It is so obvious that beginners often overlook it, but reasoning about sequence (what state exists *after* each line) is the mental skill at the heart of structured programming. In multi-threaded code, sequence within

one thread is preserved but interleaving between threads is not — that is a whole further topic (see 1.1 multi-core and later on concurrency).

```
total = 0
total = total + price      # runs first
total = total + tax        # runs second, sees updated 'total'
print(total)              # runs third
```

Selection / Branching

Selection chooses one of several execution paths depending on a Boolean condition.

- **if / else if (elif) / else** — the general form. Conditions are tested in order; the first true branch runs and the rest are skipped.
- **switch / case** — selects a branch based on the value of a single expression. Often compiled to a more efficient *jump table* than equivalent **if/elif** chains.

```
IF score >= 70 THEN
    grade = "A"
ELSE IF score >= 60 THEN
    grade = "B"
ELSE IF score >= 50 THEN
    grade = "C"
ELSE
    grade = "U"
ENDIF

SWITCH day:
    CASE "Mon": open = 9
    CASE "Sat", "Sun": open = 10
    DEFAULT:    open = 8
ENDSWITCH
```

Exam Tip: In pseudocode, examiners accept both **ELSEIF** and **ELSE IF**. Always close with **ENDIF** / **ENDSWITCH** to avoid ambiguity in nested conditionals.

Iteration

Iteration repeats a block of statements. Two main forms:

Type	Pseudocode	Use when...
Count-controlled (<code>for</code>)	<pre>FOR i ← 0 TO 9 ... NEXT i</pre>	Number of iterations known in advance (e.g. process every element of an array).
Condition-controlled pre-test (<code>while</code>)	<pre>WHILE cond DO ... ENDWHILE</pre>	May execute zero times; condition tested before entry.
Condition-controlled post-test (<code>do-while</code> / <code>repeat-until</code>)	<pre>REPEAT ... UNTIL cond</pre>	Must execute at least once (e.g. input validation).

```
# Count-controlled
FOR i ← 1 TO 10
    OUTPUT i * i
NEXT i

# Pre-test
WHILE NOT eof(file) DO
    line ← readLine(file)
    process(line)
ENDWHILE

# Post-test – guarantees at least one prompt
REPEAT
    OUTPUT "Enter 1-10:"
    n ← INPUT
UNTIL n >= 1 AND n <= 10
```

Break and continue. `break` exits the innermost loop immediately; `continue` (sometimes `next`) skips the rest of the current iteration and tests the loop condition again. Overuse can hurt readability but they are invaluable for early-exit searches.

Nested loops. A loop inside another loop. If the outer runs m times and the inner n , the inner body runs $m \times n$ times — this quickly leads to $O(n^2)$ or worse complexity. Common in grid/matrix work.

```
FOR row ← 0 TO 9
  FOR col ← 0 TO 9
    IF grid[row][col] = target THEN
      OUTPUT row, col
      BREAK          # exits inner loop only
    ENDIF
  NEXT col
NEXT row
```

Variables, Constants and Assignment

A **variable** is a named location in memory whose value can change at run time. A **constant** is a named value fixed at declaration and cannot be reassigned. **Assignment** stores a value in a variable, written `x ← expr` in pseudocode or `x = expr` in most languages.

Using constants (`PI = 3.14159` , `MAX_USERS = 500`) improves readability and maintainability: changing a single declaration updates every use, and the compiler prevents accidental modification.

Scope and Lifetime

The **scope** of an identifier is the region of source code where that name is visible and usable. The **lifetime** of a variable is the period during execution when memory is allocated for it.

Kind	Scope	Lifetime	Notes
Global	Whole program (every module/function)	Entire program execution	Easy to read/write from anywhere — but creates hidden dependencies and makes testing hard.

Local	Only inside the block/subroutine where declared	From entry to exit of that call (on the call stack)	Safer, supports re-entrancy and recursion; different calls get independent copies.
Static local	Local to a function	Entire program execution — retains value between calls	Less common at A-Level but mentioned in some textbooks.

Exam Tip: A classic exam question gives a snippet with a variable named the same inside and outside a function and asks what is printed. Answer: the local variable *shadows* the global inside the function, but outside the function the global is unchanged (unless `global` / `nonlocal` is used in Python, or reference semantics in other languages).

Procedures vs Functions

A **procedure** (sometimes called a *subroutine* or *void function*) is a named block of code that performs an action but does not return a value. A **function** is a named block that computes and **returns** a value, and ideally has no side effects.

Aspect	Procedure	Function
Returns value?	No (or implicit nothing/void)	Yes — explicit <code>return</code>
Called as...	A statement: <code>drawLine(x,y)</code>	Part of an expression: <code>a = sqrt(n) + 1</code>
Typical purpose	Perform I/O, mutate state, orchestrate other calls	Compute a derived value from inputs
Side effects	Usually — that's the point	Ideally none (pure function)

Parameters: By-Value vs By-Reference

Pass-by-value: the called routine receives a *copy* of the argument. Changes to the parameter do not affect the caller's variable. **Pass-by-reference:** the called routine receives a reference (address) to the caller's variable; assignments propagate back to the caller.

```

PROCEDURE doubleIt(BYVAL x)      # x is a copy
    x ← x * 2                    # caller sees no change
ENDPROCEDURE

PROCEDURE doubleRef(BYREF y)     # y is a reference
    y ← y * 2                    # caller's variable is updated
ENDPROCEDURE

n ← 5
doubleIt(n)                      # n is still 5
doubleRef(n)                     # n is now 10

```

Many modern languages (Java, Python) pass *object references by value*: the reference is copied but both names refer to the same underlying object, so mutations through the reference are visible but reassignment is not.

Return Values

A function uses `return` to end execution and deliver a value. Multiple exit points are permitted but some style guides prefer a single exit for clarity. A function can return compound data (tuples, arrays, objects) to effectively return “more than one thing”.

```

FUNCTION divmod(a, b)
    IF b = 0 THEN
        RETURN (0, 0)          # early exit guards division by zero
    ENDIF
    q ← a DIV b
    r ← a MOD b
    RETURN (q, r)
ENDFUNCTION

```

Modularity

Modularity is the principle of dividing a program into independent, self-contained units (**modules, functions, classes**) each with a single, well-defined responsibility and a clear interface to the rest of the system.

Benefits of modular design:

- **Reuse** — the same module can be called from many places (one correct implementation of `sort()` rather than many copies).
- **Maintainability** — bugs are confined to one module; fixes do not ripple.
- **Testability** — small modules can be unit-tested in isolation with mocked dependencies.
- **Parallel development** — different team members can work on different modules simultaneously once interfaces are agreed.
- **Abstraction** — callers need know only *what* a module does, not *how*.
- **Comprehension** — small named units make the program read like a table of contents.

Good modular design aims for **high cohesion** (each module does one thing well) and **low coupling** (modules depend on each other as little as possible).

IDE Features

An **Integrated Development Environment (IDE)** is a software application providing a unified workspace with editor, compiler/interpreter, debugger and supporting tools, aimed at raising programmer productivity.

Feature	What it does	Why it matters
Syntax highlighting	Colours keywords, strings, comments, identifiers	Makes structure instantly readable; typos jump out (e.g. unclosed string).
Autocomplete / IntelliSense	Suggests identifiers, method signatures, parameters as you type	Reduces typos, surfaces APIs, shortens typing.

Breakpoints	Pause execution at a chosen line	Inspect state at exactly the point of interest.
Step into / over / out	Execute one statement at a time, entering or skipping calls	Trace logic through branches and recursion.
Watch window	Evaluate and re-evaluate named expressions while paused	Observe how a value changes across stops.
Call stack view	Shows the chain of active function calls	Essential for recursion and diagnosing crashes.
Linters / static analysis	Flags style problems and likely bugs without running code	Catches unused variables, shadowed names, unreachable code.
Refactoring tools	Automated rename, extract method, inline, move	Safe, mechanical transformations that preserve behaviour.
Version control integration	Git commits, diffs, blame, branch switching in the editor	History at your fingertips; easy bisecting and merge-conflict resolution.
Build / run / test runners	One-click compile, launch, run unit tests	Tight feedback loop — the secret of productive coding.

Debugging Techniques

Debugging is the disciplined activity of *locating* and *removing* defects. A defect (bug) is a difference between expected and actual behaviour. Techniques range from primitive to precise:

1. **Print/logging ("printf debugging")** — insert `print` or structured `log.debug` calls to dump variable values at key points. Quick and language-neutral. Downsides: editing code each time, noisy output, easy to forget to remove.
2. **Breakpoints** — click a margin in the IDE to pause execution when a line is reached. Can be conditional ("break only when `i == 50`").
3. **Step-through** — advance execution one line at a time. *Step into* enters a called function; *step over* runs the call as a unit; *step out* runs to the end of the current function.

4. **Watch expressions** — continuously display the value of an expression (e.g. `len(queue)`) each time execution halts.
5. **Call stack inspection** — view the chain of pending calls; click any frame to inspect its local variables. Vital for recursion.
6. **Bisecting with version control** — `git bisect` performs binary search over commits, testing each midpoint to locate the commit that introduced a regression. Turns “when did this break?” into $O(\log n)$ searches.
7. **Unit tests / regression tests** — automate reproduction; once fixed, the test prevents the bug from re-entering.

Exam Tip: Examiners often ask you to compare debugging methods. A strong answer pairs print-statements (quick, crude, messy) with breakpoints (precise, non-invasive, IDE-dependent). Mention conditional breakpoints to show depth.

Recursion

Recursion is a technique in which a subroutine solves a problem by calling itself on a smaller version of that problem. Every recursive definition must include a **base case** (the smallest instance, solved directly, no further call) and at least one **recursive (general) case** that reduces toward the base case.

Why two cases are required: without a base case, recursion never stops and the program crashes with a *stack overflow*. Without a recursive case, the algorithm only handles the trivial input. Together they define an inductive process: solve the smallest case, then express bigger cases in terms of smaller ones.

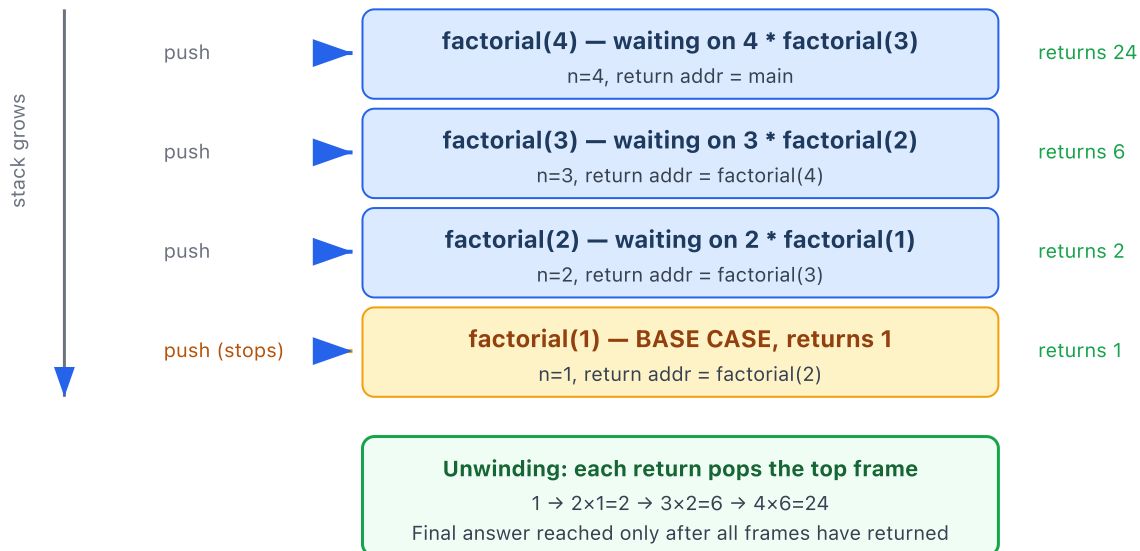
Stack Frames and Stack Overflow

A **stack frame** (activation record) is the block of memory pushed onto the **call stack** for each active subroutine invocation. It stores the return address, parameters, local variables and saved registers. When the call returns, its frame is popped.

Every recursive call pushes a new frame. If recursion is too deep (e.g. `factorial(1000000)` in Python) the stack's reserved memory is exhausted and the run-time raises *stack overflow* / `RecursionError`. Typical defaults are 1–10 MB of stack, allowing a few thousand levels.

Call Stack during factorial(4)

Frames grow downwards as calls are made, then unwind upwards as each returns



Worked Example 1: Factorial

Definition: $n! = n \times (n-1) \times \dots \times 2 \times 1$; and $0! = 1$.

```
# Recursive
FUNCTION factorial(n)
    IF n <= 1 THEN                # base case
        RETURN 1
    ELSE
        RETURN n * factorial(n - 1)  # recursive case
    ENDIF
ENDFUNCTION

# Iterative equivalent
FUNCTION factorialIter(n)
    result ← 1
    FOR i ← 2 TO n
        result ← result * i
    NEXT i
```

```

    RETURN result
ENDFUNCTION

```

Both are $O(n)$ time. The recursive version uses $O(n)$ additional stack memory for frames; the iterative uses $O(1)$.

Worked Example 2: Fibonacci — Naive vs Memoised

Fibonacci sequence: $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$.

```

# Naive recursion – exponential  $O(2^n)$  because it re-computes the same
# subproblems many times (e.g.  $F(5)$  calls  $F(3)$  twice,  $F(2)$  three times...)
FUNCTION fib(n)
    IF n < 2 THEN RETURN n
    RETURN fib(n - 1) + fib(n - 2)
ENDFUNCTION

# Memoised: store previously computed results –  $O(n)$  time,  $O(n)$  space
memo ← new Dictionary()

FUNCTION fibMemo(n)
    IF n < 2 THEN RETURN n
    IF memo.contains(n) THEN
        RETURN memo[n]                # cache hit – avoid recomputation
    ENDIF
    result ← fibMemo(n - 1) + fibMemo(n - 2)
    memo[n] ← result
    RETURN result
ENDFUNCTION

# Iterative  $O(n)$  time,  $O(1)$  space – best for large n
FUNCTION fibIter(n)
    a ← 0 ; b ← 1
    FOR i ← 1 TO n
        (a, b) ← (b, a + b)
    NEXT i
    RETURN a
ENDFUNCTION

```

Running `fib(40)` on a modern laptop takes around a second; the memoised version is instant. The naive call tree for `fib(5)` has 15 nodes; for `fib(40)` it has over 300 million. The memoised

version computes each of $F(0) \dots F(n)$ exactly once.

Approach	Time	Space	Notes
Naive recursion	$O(2^n)$	$O(n)$ stack	Recomputes massively; fine for teaching, hopeless beyond $n \approx 35$.
Memoised recursion	$O(n)$	$O(n)$ cache + stack	Each $F(k)$ computed once; ideal when other results also needed.
Bottom-up iterative	$O(n)$	$O(1)$	Best when you only need $F(n)$; no cache, no stack.
Matrix exponentiation	$O(\log n)$	$O(1)$	Uses $[[1,1],[1,0]]^n$; out-of-scope but worth noting.

Worked Example 3: Towers of Hanoi

Move n disks from peg A to peg C using peg B , never placing a larger disk on a smaller one.

```

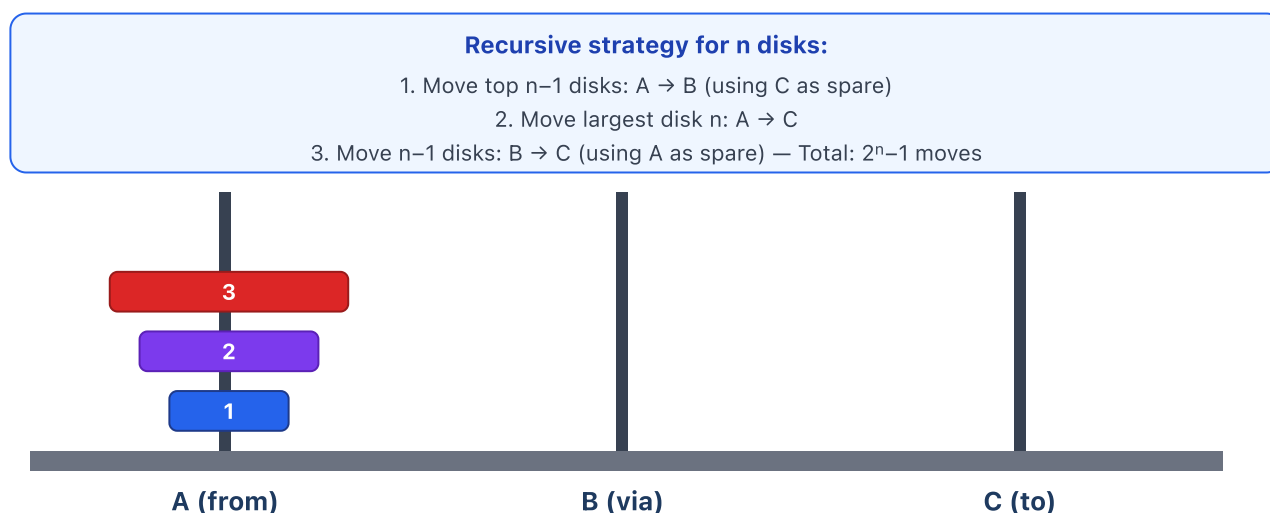
PROCEDURE hanoi(n, from, via, to)
  IF n = 1 THEN
    OUTPUT "Move disk 1 from " + from + " to " + to    # base case
  ELSE
    hanoi(n - 1, from, to, via)                        # 1. move n-1 aside
    OUTPUT "Move disk " + n + " from " + from + " to " + to  # 2. move biggest
    hanoi(n - 1, via, from, to)                        # 3. bring n-1 back
  ENDIF
ENDPROCEDURE

hanoi(3, "A", "B", "C")

```

Total moves = $2^n - 1$ (7 moves for $n=3$, 1023 for $n=10$). The algorithm is provably optimal.

Towers of Hanoi — starting state (n=3)



Each sub-move of $n-1$ disks is itself a smaller Hanoi problem — perfect recursion.

Worked Example 4: In-order Binary Tree Traversal

A binary search tree (BST) stores smaller keys in the left subtree, larger in the right. An *in-order* traversal visits left $\rightarrow$ node $\rightarrow$ right, producing sorted output.

```

PROCEDURE inOrder(node)
    IF node = NULL THEN RETURN           # base case: empty subtree
    inOrder(node.left)                  # recurse left
    OUTPUT node.value                    # visit
    inOrder(node.right)                 # recurse right
ENDPROCEDURE

```

Trivially extends to pre-order (visit before) and post-order (visit after). Natural fit for recursion because a tree is itself a recursive data structure.

Recursion vs Iteration

Aspect	Recursion	Iteration
Code clarity	Often closer to the mathematical/structural definition (trees, fractals, divide-and-conquer)	Simpler for linear processes (sums, counts, loops over arrays)

Memory	One stack frame per call — $O(\text{depth})$ extra memory	Constant memory (one loop counter)
Speed	Call overhead per invocation	Slightly faster; no call overhead
Risk	Stack overflow on deep input; may recompute subproblems	No stack overflow; easy to write infinite loops if condition wrong
Termination	Must hit base case	Must eventually falsify loop condition

Converting recursion to iteration is always possible: replace the implicit call stack with an explicit `Stack` data structure and loop until it is empty. Example — iterative DFS in place of recursive:

```

PROCEDURE dfsIter(root)
    stack ← new Stack()
    stack.push(root)
    WHILE NOT stack.isEmpty()
        node ← stack.pop()
        IF node ≠ NULL THEN
            visit(node)
            stack.push(node.right)  # pushed first so left is visited first
            stack.push(node.left)
        ENDIF
    ENDWHILE
ENDPROCEDURE

```

Tail Recursion

A call is **tail recursive** if it is the *last* action performed by the function — nothing remains to be computed after the call returns. Some compilers (Scheme, Scala) apply **tail-call optimisation (TCO)**: they reuse the current stack frame instead of pushing a new one, reducing space to $O(1)$ and avoiding stack overflow.

```
# Factorial rewritten in tail-recursive form (accumulator pattern)
FUNCTION factorialTR(n, acc = 1)
  IF n <= 1 THEN RETURN acc
  RETURN factorialTR(n - 1, acc * n)      # tail call – nothing follows
ENDFUNCTION
```

Python, Java and C# do *not* perform TCO, so tail recursion there is mainly a stylistic choice. C/C++ compilers often apply it as an optimisation.

Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming paradigm in which programs are organised around **objects** — bundles of **state** (attributes/fields) and **behaviour** (methods) — that interact by sending messages (method calls).

Classes, Objects and Instances

Term	Meaning
Class	A blueprint/template describing the attributes and methods that objects of that kind will share.
Object	A concrete entity created at run time from a class; has its own copy of the attributes.
Instance	Synonym for object, emphasising the class it was instantiated from ("d is an instance of Dog").
Attribute / field	A variable that belongs to an object (or class, if static).
Method	A function that belongs to an object (or class, if static).

Constructors and Destructors

A **constructor** is a special method, invoked automatically when an object is created, that initialises the new object's state. In most pseudocode it is `new` or `__init__`. A **destructor** runs when the object is destroyed — used to release resources (files, network connections, memory). In garbage-collected languages destructors are rarely used; C++ makes heavy use of them (RAII).

Static vs Instance Members

Member	Belongs to	Accessed via	Example
Instance	A specific object; each object has its own copy	The object: <code>d.name</code>	Every <code>Dog</code> has its own <code>name</code> .
Static / class	The class itself; one copy shared by all instances	The class: <code>Dog.totalDogs</code>	A counter of how many <code>Dog</code> objects exist.

The Four Pillars of OOP

Pillar 1 — Encapsulation

Encapsulation is the bundling of state and the methods that operate on that state into a single unit (an object), combined with **information hiding**: restricting direct access to the internals so they can only be manipulated through a controlled public interface.

Access modifiers enforce encapsulation:

Modifier	Meaning
<code>public</code>	Accessible from anywhere.
<code>private</code>	Accessible only from within the same class.
<code>protected</code>	Accessible from the class and its subclasses.

Getters and setters (accessor/mutator methods) provide controlled access: the setter can validate input, the getter can transform or deny based on state. Clients talk only to the interface,

so the class can change its internal representation without breaking callers.

Pillar 2 — Inheritance

Inheritance lets a *child class* (subclass, derived class) acquire the attributes and methods of a *parent class* (superclass, base class). The child extends or specialises the parent. This creates a class hierarchy representing an “**is-a**” relationship: *a Dog is an Animal*.

- **Override** — a child class can provide its own implementation of an inherited method to specialise behaviour.
- **super()** — calls the parent’s version of a method (e.g. to run the parent constructor first, then add subclass setup).
- **Single vs multiple inheritance** — some languages (Java, C#) allow only one parent (plus interfaces); others (Python, C++) allow multiple parents, which can cause the *diamond problem*.

Pillar 3 — Polymorphism

Polymorphism (“many forms”) is the ability for one interface to represent different underlying types. A variable typed as a parent class can refer to any of its subclasses, and calling a method selects the correct subclass’s version at run time.

Kind	Binding	Example
Overloading	Compile-time (static) — chosen by parameter signature	<code>add(int, int)</code> vs <code>add(float, float)</code> vs <code>add(String, String)</code> .
Overriding	Run-time (dynamic) — chosen by the object’s actual class	<code>Animal a = new Dog(); a.speak();</code> calls <code>Dog.speak</code> .

Interfaces (Java, C#) or abstract methods (Python’s `ABC` , C++’s pure virtual) let unrelated classes share a common calling convention.

Liskov Substitution Principle (LSP). A program that uses a parent type should work correctly if any subclass is substituted. Subclasses must not strengthen preconditions or weaken postconditions.

Pillar 4 — Abstraction

Abstraction is the act of exposing only the essential features of an object while hiding the implementation details. In OOP this is achieved using **abstract classes** (partially implemented; cannot be instantiated) and **interfaces** (pure contract — signatures only, no code).

Encapsulation hides *data*; abstraction hides *implementation*. The two are complementary — good OOP uses both.

Device	Can be instantiated?	Can contain code?	Purpose
Concrete class	Yes	Yes, fully	Normal working class.
Abstract class	No — only subclasses can	Yes, partially (some methods abstract, others implemented)	Shared state + partial behaviour for a family of subclasses.
Interface	No	Signatures only (default methods in modern Java/C#)	Pure contract; multiple inheritance of behaviour in Java/C#.

When to choose which. Use an abstract class when subclasses genuinely share state and partial behaviour (the “is-a-kind-of” relationship). Use an interface when unrelated classes must merely obey the same calling convention (the “can-do” relationship — `Sortable` , `Printable` , `Comparable`).

Worked Example — Animal Hierarchy Showing All Four Pillars

```
ABSTRACT CLASS Animal
    PRIVATE name
    PROTECTED age

    CONSTRUCTOR(n, a)
        name ← n
        age ← a
    ENDCONSTRUCTOR

    PUBLIC FUNCTION getName()
        RETURN name
    ENDFUNCTION

    PUBLIC PROCEDURE setName(n)
        IF length(n) > 0 THEN name ← n
    ENDPROCEDURE

    PUBLIC ABSTRACT PROCEDURE speak()
ENDCLASS

CLASS Dog INHERITS Animal
    PRIVATE breed

    CONSTRUCTOR(n, a, b)
        SUPER(n, a)
        breed ← b
    ENDCONSTRUCTOR

    PUBLIC PROCEDURE speak()
        OUTPUT getName() + " says Woof!"
    ENDPROCEDURE
ENDCLASS

CLASS Cat INHERITS Animal
    PUBLIC PROCEDURE speak()
        OUTPUT getName() + " says Meow!"
    ENDPROCEDURE
ENDCLASS

# POLYMORPHISM in action
animals ← [new Dog("Rex", 3, "Labrador"), new Cat("Luna", 5)]
FOR EACH a IN animals
```

ABSTRACTION – cannot instantiate
ENCAPSULATION – hidden field

getter – controlled access

setter – can validate

must be overridden

INHERITANCE

call Animal's constructor

OVERRIDE – polymorphism

different OVERRIDE

```
a.speak()      # correct subclass method chosen at run time
NEXT a
```

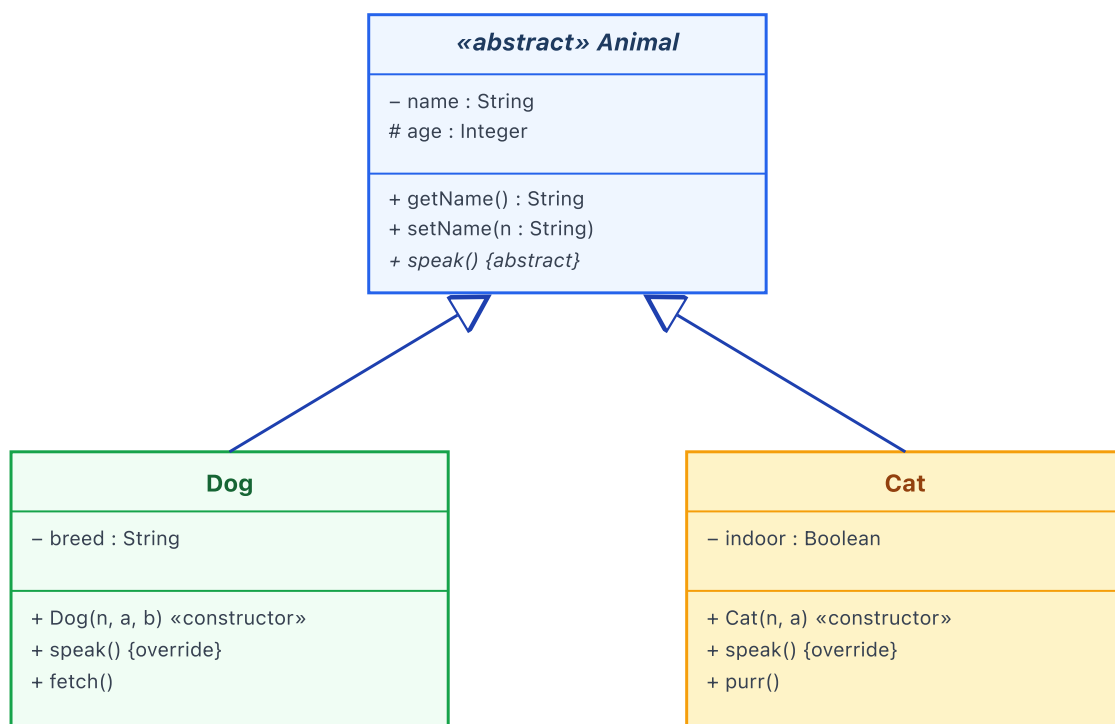
Output:

```
Rex says Woof!
Luna says Meow!
```

This tiny example illustrates **all four pillars**: data is *encapsulated* with getters/setters, *inheritance* produces `Dog` and `Cat` from `Animal`, *polymorphism* lets the loop call the correct `speak()`, and *abstraction* is achieved by making `Animal` abstract with an unimplemented `speak()`.

UML Class Diagram

UML Class Diagram — Animal Hierarchy



Key: `-` private `#` protected `+` public `«...»` stereotype hollow triangle = inheritance ("is-a")

Advantages and Disadvantages of OOP

Advantages	Disadvantages
------------	---------------

Models real-world domains naturally (students, invoices, animals)	More boilerplate — short scripts become verbose
Reuse via inheritance and composition	Inheritance overused leads to fragile, rigid hierarchies
Encapsulation improves maintainability	Indirection and dynamic dispatch have a small performance cost
Polymorphism enables extensible, open/closed designs	Steep learning curve; easy to misuse (God objects, tight coupling)
Large teams can divide work by class	Debugging call graphs across many classes can be hard

When OOP Is (and Isn't) the Right Tool

Paradigm	Best for	Example languages
OOP	Long-lived systems with complex domain models — GUIs, games, business apps	Java, C#, C++, Python
Procedural	Short scripts, low-level system code, straightforward data transformations	C, early Pascal, shell
Functional	Heavy parallelism, mathematical transformations, when side-effects are dangerous	Haskell, F#, Scala, Clojure

Exam Tip: If an exam asks when OOP is *not* appropriate, mention: tiny scripts (too much overhead), hot inner loops where virtual dispatch costs matter, problems that are naturally a pipeline of pure data transformations (functional fits better).

2.2.2 Computational Methods

Computational methods are the strategic toolkit for *solving* problems once they have been analysed with computational thinking (2.1). Where 2.1 asks "how should I think about this problem?", 2.2.2 asks "which algorithmic strategy fits?".

Problem Recognition

Before writing code, recognise which established class a problem belongs to:

- Searching an unsorted list → linear search.
- Searching a sorted list → binary search (divide and conquer).
- Shortest path in a weighted graph → Dijkstra / A* (see 2.3).
- Try-all-configurations with early abandonment → backtracking.
- Finding patterns in huge datasets → data mining / machine learning.
- Approximate answer needed fast → heuristic.

Recognising the pattern means you can reach for a known-good algorithm rather than reinventing one. A surprising amount of professional programming is *pattern matching* existing problems to textbook solutions.

Worked walkthrough — picking a strategy

Consider: "Given a million customer records, group them into similar-spending segments and flag unusual accounts." An experienced programmer immediately pattern-matches:

- "Group into similar-spending segments" → **clustering** (unsupervised data mining) — probably k-means.
- "Flag unusual accounts" → **anomaly detection** — distance from cluster centroids or isolation forest.
- "A million records" → need efficient $O(n)$ or $O(n \log n)$ per pass; cannot use $O(n^2)$ pairwise comparisons.
- The pipeline: extract → clean (drop nulls, normalise currencies) → feature engineer (monthly spend, recency) → cluster → detect → visualise → report.

Recognising this decomposition up-front turns a vague request into a concrete sequence of known-solvable sub-problems.

Problem Decomposition

Decomposition (introduced in 2.1) is the act of breaking a complex problem into smaller, independently solvable sub-problems. Each sub-problem becomes a module/function with a clear input/output contract.

Decomposition is the opposite face of modularity: modularity is about how the finished code is *structured*; decomposition is the *thought process* that gets you there. The end result of good decomposition is a program that reads like a table of contents.

Divide and Conquer

Divide and conquer solves a problem by (1) **dividing** it into two or more smaller sub-problems of the same type, (2) **conquering** (solving) each sub-problem — usually recursively, (3) **combining** the sub-solutions into a solution for the original problem.

Algorithm	Divide	Conquer	Combine	Complexity
Binary search	Pick midpoint; compare	Search one half	None (return index)	$O(\log n)$
Merge sort	Split array in half	Sort each half	Merge sorted halves	$O(n \log n)$
Quicksort	Partition around pivot	Sort each partition	Concatenate	$O(n \log n)$ avg, $O(n^2)$ worst

Detailed implementations appear in 2.3. The pattern is instantly recognisable: the recursive case makes calls on strictly smaller inputs and the base case handles a trivial input (empty/singleton array).

Abstraction in Problem Solving

Abstraction (revisited from 2.1) is the suppression of irrelevant detail to reveal the essential structure. A tube map abstracts away geography; `Queue` abstracts away whether the queue is a

linked list or a ring buffer. Good abstractions let you reason about a problem at the level it naturally lives, and they compose (solutions built from abstract parts remain tractable as size grows).

Thinking Ahead and Caching

Thinking ahead (2.1.2) means predicting what inputs, edge cases and resources will be needed, and preparing for them. **Caching** is a specific form: store the result of an expensive computation so a repeat request returns instantly.

- **Memoisation** — function-level caching (see memoised Fibonacci above).
- **Lookup tables** — precompute all answers for a small domain (e.g. CRC tables, trig tables).
- **Browser/HTTP caches** — avoid re-downloading unchanged resources.
- **CPU caches** (1.1) — hardware form of the same idea.

Caching trades memory for speed. It is only correct if the cached answer is still valid — cache invalidation is famously described as one of the two hard problems in computing.

Backtracking

Backtracking is a refinement of brute-force search. It incrementally builds a partial solution and, at each step, tests whether the partial solution can still succeed. When it cannot, it abandons ("backtracks") and tries the next alternative. This prunes away entire branches of the search tree that need not be explored.

Classic applications: N-Queens, Sudoku solvers, maze running, parsing with ambiguous grammars, generating permutations.

Worked Example — N-Queens

Place N queens on an N×N chessboard so no two attack each other.

```
board ← array of N integers, board[row] = column of queen in that row
```

```
FUNCTION isSafe(row, col)
  FOR r ← 0 TO row - 1
    IF board[r] = col THEN RETURN FALSE           # same column
    IF abs(board[r] - col) = abs(r - row) THEN
```

```

        RETURN FALSE                                # same diagonal
    ENDIF
NEXT r
RETURN TRUE
ENDFUNCTION

FUNCTION solve(row)
    IF row = N THEN
        printBoard()
        RETURN TRUE                                # found a solution
    ENDIF
    FOR col ← 0 TO N - 1
        IF isSafe(row, col) THEN
            board[row] ← col                        # place queen – partial solution
            IF solve(row + 1) THEN RETURN TRUE
            # implicit backtrack: loop continues, overwriting board[row]
        ENDIF
    NEXT col
    RETURN FALSE                                    # no column worked – caller will backtrack
ENDFUNCTION

solve(0)

```

Worked Example — Maze Runner

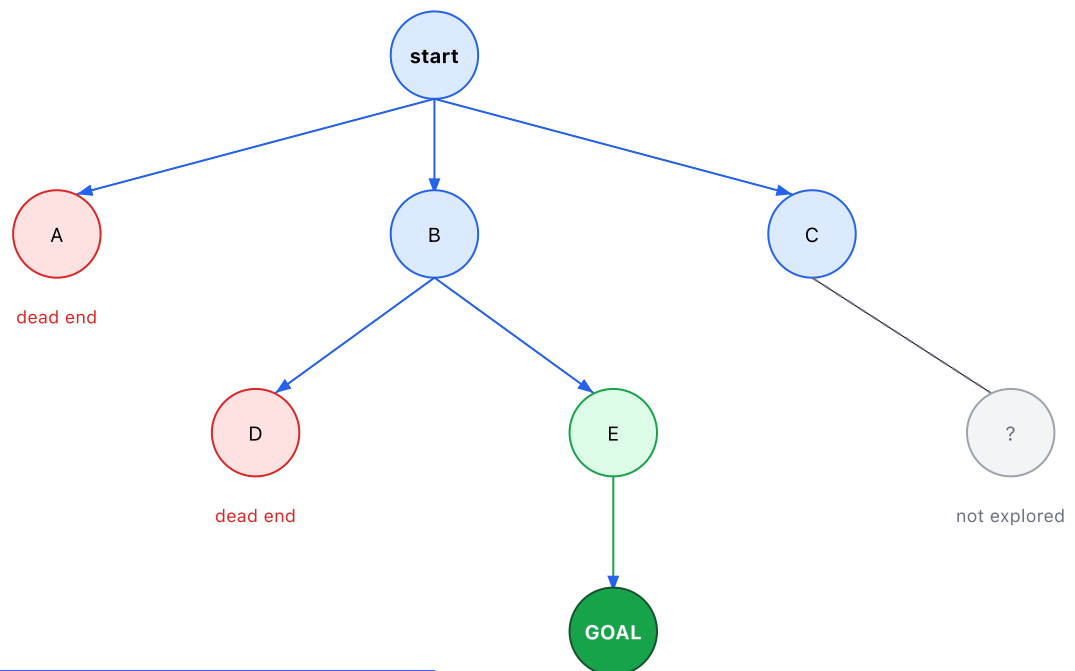
```

FUNCTION findPath(x, y)
    IF (x, y) = goal THEN RETURN TRUE
    IF NOT inBounds(x, y) OR maze[x][y] = WALL OR visited[x][y] THEN
        RETURN FALSE
    ENDIF
    visited[x][y] ← TRUE                            # commit
    FOR EACH (dx, dy) IN [(0,1), (1,0), (0,-1), (-1,0)]
        IF findPath(x + dx, y + dy) THEN
            path.push((x, y))
            RETURN TRUE
        ENDIF
    NEXT
    visited[x][y] ← FALSE                            # backtrack
    RETURN FALSE
ENDFUNCTION

```

Backtracking Decision Tree (partial)

Green = success branch, Red = dead end (backtrack)



Order explored: start → A (fail) → B → D (fail) → E → GOAL

If N-Queens were solved by pure brute force it would examine N^N boards (about 16.8 million for $N=8$). Backtracking with pruning examines only a few thousand — orders of magnitude fewer.

General structure of a backtracking algorithm:

```

FUNCTION backtrack(partialSolution)
  IF partialSolution is complete THEN
    record or return solution
  RETURN
ENDIF
FOR EACH candidate IN options(partialSolution)
  IF candidate is valid THEN
    apply(candidate)           # extend partial solution
    backtrack(partialSolution) # recurse
    undo(candidate)            # undo – “the backtrack”
  ENDIF
NEXT
ENDFUNCTION
  
```

The two crucial steps are **pruning** (rejecting candidates that cannot extend to a full solution) and **undoing** (restoring state so the next candidate starts fresh). Without pruning backtracking

degrades to brute force; without undoing, sibling branches inherit corrupt state.

Data Mining

Data mining is the process of discovering patterns, correlations and anomalies in *large* datasets, usually combining statistics, machine learning, and database systems. It transforms raw data into actionable insight.

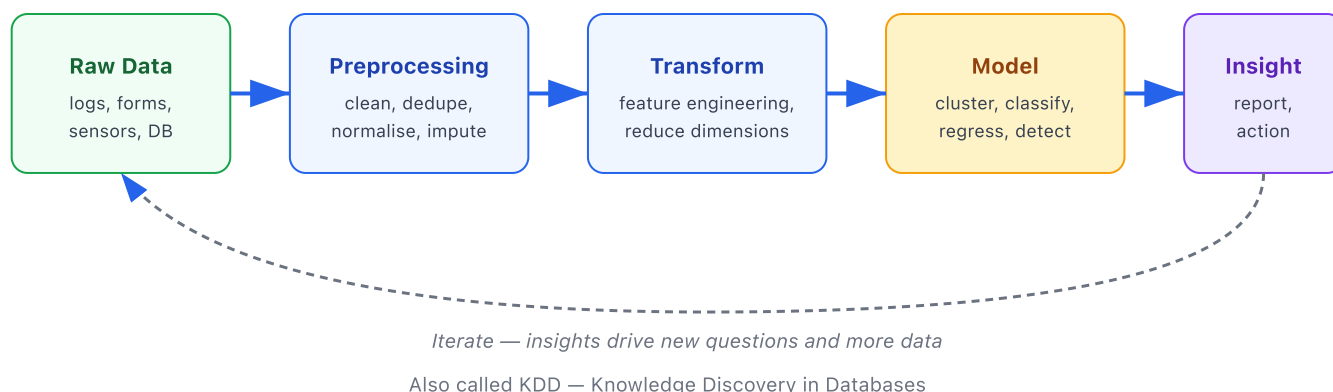
Common data mining tasks:

Task	Goal	Example
Clustering	Group similar records without pre-defined labels	Customer segmentation from purchase histories (k-means).
Classification	Assign records to a known set of categories	Spam vs not-spam; cancerous vs benign tumour.
Regression	Predict a continuous value	House price from features.
Association rule mining	Find items that co-occur	"People who buy nappies also buy beer" (market-basket analysis).
Anomaly detection	Flag records that don't fit the pattern	Credit-card fraud, network intrusion.

Applications:

- **Retail** — stock levels, product placement, recommendation engines.
- **Finance** — fraud detection, credit scoring, algorithmic trading.
- **Healthcare** — diagnosis support, epidemiology.
- **Science** — genomics, astrophysics, particle physics.
- **Social media** — friend suggestions, ad targeting.

The Data Mining Pipeline



Heuristics

A **heuristic** ("rule of thumb") is a pragmatic technique that produces a *good enough* answer quickly, sacrificing guarantees of optimality for speed or simplicity. Used when an exact algorithm would be too slow, or no exact algorithm is known.

Application	Heuristic	Trade-off
Pathfinding (A*)	$h(n)$ = straight-line distance to goal	May not find the absolute shortest path if h overestimates.
Spam filtering	Keyword score + sender reputation + Bayesian update	Occasional false positives/negatives; much faster than reading every email.
Travelling salesman	Nearest neighbour; 2-opt; simulated annealing	Near-optimal tour in seconds vs exact exponential algorithms.
Chess engines	Piece values, mobility, king safety	Can't evaluate every position to checkmate; heuristic guides pruning.
File compression	Huffman frequency estimation	May not be the theoretically optimal code length for the true distribution.

Exam Tip: A good heuristic is *admissible* (never over-estimates cost, for A*), *consistent* (triangle inequality), and *cheap to compute*. Examiners love the contrast: heuristic = fast + approximate; exhaustive = slow + optimal.

Performance Modelling

Performance modelling predicts and measures how a program's resource use (time, memory, power, bandwidth) scales with input size. It answers: "Will this still be fast enough when the input grows 1000x?"

- **Analytical** (Big-O, see 2.3) — predict asymptotic growth on paper before coding.
- **Profiling** — run the program with instrumentation that records where time and memory are spent (e.g. `cProfile`, `perf`, Visual Studio Profiler). Identifies the *hot spots* worth optimising.
- **Benchmarking** — time the program on representative workloads under controlled conditions.
- **Stress / load testing** — push inputs or users well beyond expected, to find breaking points.

Performance is almost always a *time/space trade-off*. Caching, memoisation and lookup tables spend memory to save time. Compression and streaming spend time to save memory or bandwidth. The right point on the curve depends on the constraints of the deployment environment (server farm vs embedded device).

Pipelining (Software)

Software pipelining structures a computation as a series of processing stages, each transforming the output of the previous stage. Data flows through like items on an assembly line. Stages can run **concurrently** on different items, improving throughput.

Distinct from the *CPU pipelining* of 1.1 (fetch/decode/execute at the hardware level) but conceptually identical. Examples:

- **Unix pipes:** `cat logs | grep ERROR | sort | uniq -c` — four processes run simultaneously, each consuming what the previous emits.
- **ETL pipelines** (Extract, Transform, Load) in data engineering.

- **Build pipelines** — compile → test → package → deploy, with caching between stages.
- **Graphics pipelines** — vertices → transforms → rasterise → shade → blend.
- **Machine-learning pipelines** (scikit-learn Pipeline) — pre-process → feature-select → model → evaluate.

Benefits: modularity (each stage tested separately), parallelism (stages run on different cores/machines), streaming (don't need the full dataset in memory at once). **Drawbacks:** bottleneck stage limits whole pipeline throughput; debugging across stage boundaries can be harder; error handling is more complex when data has already flowed downstream.

Visualisation

Visualisation is the representation of data or program state in graphical form — charts, diagrams, dashboards — to reveal patterns and outliers that are invisible in raw tables.

Good visualisations are a computational method in their own right: an analyst can spot a cluster or spike in a scatter plot in seconds, whereas reading the underlying CSV would take hours. Choose the form to match the question:

Form	Good for
Bar / column chart	Comparing category totals
Line chart	Trends over time
Scatter plot	Correlation between two variables, outliers
Heatmap	Density, 2D distributions, correlation matrices
Histogram / box plot	Distribution of a single variable
Network / graph diagram	Relationships between entities
Dashboard	Multiple correlated views updated live
UML / flowchart	Structure of code or process

Visualisation is also central to **debugging and profiling**: flame graphs, call-graph views, memory heat-maps and animation of data structures all use visual channels (position, colour, size) to compress information.

Principles of effective visualisation (Tufte, Few, Munzner):

- **Data-to-ink ratio** — every mark on the chart should carry information. Remove chartjunk (gratuitous gradients, 3-D effects).
- **Accurate encoding** — quantitative differences should be shown by length or position, not area or colour intensity, which are harder to compare.
- **Axis honesty** — don't truncate a y-axis to exaggerate a difference; don't use dual axes without clear labelling.
- **Colour for category, not quantity** — unless a proper sequential / diverging palette is used. Always check colour-blind safety.
- **Small multiples** — many small copies of the same chart with one variable changing is often clearer than one busy chart.

Exam Tip: Whenever an exam question invites a worked answer with data or structures, a quick diagram (even a rough sketch) often scores the same marks as a paragraph — and is faster to produce.

Bringing 2.1 Concepts Back Together

2.2.2 is best viewed as the applied face of 2.1's computational thinking. The same ideas return with sharper teeth:

2.1 concept	Reappears in 2.2.2 as...
Abstraction	Hiding implementation in classes; picking the right level for a problem.
Decomposition	Dividing into modules, functions, pipeline stages.
Thinking ahead	Caching, memoisation, heuristics with lookahead.
Thinking procedurally	Programming constructs, subroutines.

Thinking logically	Selection, backtracking conditions, invariants.
Thinking concurrently	Pipelining, divide-and-conquer on multiple cores.

Common exam traps

- Confusing **overloading** (same name, different signatures, chosen at compile time) with **overriding** (subclass replaces a parent method, chosen at run time).
- Forgetting that a procedure call with **BYREF** parameters can mutate the caller's variables — a frequent cause of exam trace-table errors.
- Writing recursion without a **base case** — an automatic zero-mark in an exam solution.
- Describing data mining purely as "looking for patterns" — you need to name concrete tasks (clustering, classification, anomaly detection).
- Confusing *software pipelining* (this topic) with the *CPU pipelining* from 1.1.

Summary Checklist

- Three constructs: sequence, selection, iteration (for/while/do-while) — with break, continue, nesting.
- Variables vs constants; scope (global/local) and lifetime; pass-by-value vs pass-by-reference.
- Procedures return nothing; functions return a value.
- Modularity benefits: reuse, maintainability, testability, parallel development, abstraction.
- IDE features: syntax highlighting, autocomplete, breakpoints, stepping, watches, call stack, linting, refactoring, VCS.
- Debugging hierarchy: print, breakpoints, stepping, watches, call stack, bisect.
- Recursion: **base case + recursive case**; stack frames; stack overflow; factorial, Fibonacci (naive vs memoised), Hanoi, tree traversal; recursion $\leftrightarrow$ iteration; tail recursion.
- OOP four pillars: **encapsulation, inheritance, polymorphism, abstraction**. Classes/objects, constructors/destructors, static vs instance, overloading vs overriding, Liskov.
- Computational methods: problem recognition, decomposition, divide and conquer, abstraction, thinking ahead/caching, backtracking, data mining, heuristics, performance modelling,

pipelining, visualisation.